Q2. [9 pts] A Miscellany

(a) [3 pts] Consider a convolutional neural network for reading the handwritten MNIST letters, which are 28×28 images. Suppose the first hidden layer is a convolutional layer with 20 different 5×5 filters, applied to the input image with a stride of 1 (i.e., every filter is applied to *every* 5×5 patch of the image, with patches allowed to overlap). Each filter has a bias weight. How many weights (parameters) does this layer use?

$20 \times (5 \times 5 + 1) = 520$.

(b) [3 pts] Let X be an $n \times d$ design matrix representing n sample points in $\mathbb{R}^d$. Let $X = UDV^\top$ be the singular value decomposition of X. We stated in lecture that row i of the matrix UD gives the coordinates of sample point X_i in principal coordinates space, i.e., $X_i \cdot v_j$ for each j, where X_i is the ith row of X and v_j is the jth column of V. Show that this is true.

As V is an orthogonal matrix, we can write $XV = UDV^\top V = UD$.

By the definition of matrix multiplication, $(UD)_{ij} = X_i \cdot v_j$.

(c) [3 pts] Let $x, y \in \mathbb{R}^d$ be two points (e.g., sample or test points). Consider the function $k(x, y) = x^\top \text{rev}(y)$ where rev(y) reverses the order of the components in y. For example, $\text{rev}\left(\begin{bmatrix}1\\2\\3\end{bmatrix}\right) = \begin{bmatrix}3\\2\\1\end{bmatrix}$. Show that k *cannot* be a valid kernel function.

Hint: remember how the kernel function is defined, and show a simple two-dimensional counterexample.

We have that $k((-1, 1), (-1, 1)) = -2$, but this is impossible as, if k is a valid kernel, then there is some function Φ such that $k(x, x) = \Phi(x)^\top \Phi(x) \geq 0$.

Q3. [10 pts] Maximum Likelihood Estimation for Reliability Testing

Suppose we are reliability testing n units taken randomly from a population of identical appliances. We want to estimate the mean failure time of the population. We assume the failure times come from an exponential distribution with parameter $\lambda > 0$, whose probability density function is $f(x) = \lambda e^{-\lambda x}$ (on the domain $x \geq 0$) and whose cumulative distribution function is $F(x) = \int_0^x f(x)\,\mathrm{d}x = 1 - e^{-\lambda x}$.

(a) [6 pts] In an ideal (but impractical) scenario, we run the units until they all fail. The failure times are $t_1, t_2, \ldots, t_n$.

Formulate the likelihood function $\mathcal{L}(\lambda; t_1, \ldots, t_n)$ for our data. Then find the maximum likelihood estimate $\hat{\lambda}$ for the distribution's parameter.

$$\begin{aligned}
\mathcal{L}(\lambda; t_1, \ldots, t_n) &= \prod_{i=1}^{n} f(t_i) = \prod_{i=1}^{n} \lambda e^{-\lambda t_i} = \lambda^n e^{-\lambda \sum_{i=1}^{n} t_i} \\
\ln \mathcal{L}(\lambda) &= n \ln \lambda - \lambda \sum_{i=1}^{n} t_i \\
\frac{\partial}{\partial \lambda} \ln \mathcal{L}(\lambda) &= \frac{n}{\lambda} - \sum_{i=1}^{n} t_i = 0 \\
\hat{\lambda} &= \frac{n}{\sum_{i=1}^{n} t_i}
\end{aligned}$$

(b) [4 pts] In a more realistic scenario, we run the units for a fixed time T. We observe r unit failures, where $0 \leq r \leq n$, and there are $n - r$ units that survive the entire time T without failing. The failure times are $t_1, t_2, \ldots, t_r$.

Formulate the likelihood function $\mathcal{L}(\lambda; n, r, t_1, \ldots, t_r)$ for our data. Then find the maximum likelihood estimate $\hat{\lambda}$ for the distribution's parameter.

Hint 1: What is the probability that a unit will not fail during time T? *Hint 2:* It is okay to define $\mathcal{L}(\lambda)$ in a way that includes contributions (densities and probability masses) that are not commensurate with each other. Then the constant of proportionality of $\mathcal{L}(\lambda)$ is meaningless, but that constant is irrelevant for finding the best-fit parameter $\hat{\lambda}$. *Hint 3:* If you're confused, for part marks write down the likelihood that r units fail and $n-r$ units survive; then try the full problem. *Hint 4:* If you do it right, $\hat{\lambda}$ will be the number of observed failures divided by the sum of unit test times.

$$\begin{aligned}
\mathcal{L}(\lambda; n, r, t_1, \ldots, t_r) &\propto \left(\prod_{i=1}^{r} f(t_i) \right) (1 - F(T))^{n-r} \\
&= \left(\prod_{i=1}^{r} \lambda e^{-\lambda t_i} \right) \left(e^{-\lambda T} \right)^{n-r} \\
&= \lambda^r e^{-\lambda \sum_{i=1}^{r} t_i} e^{-\lambda (n-r) T} \\
\ln \mathcal{L}(\lambda) &= r \ln \lambda - \lambda \sum_{i=1}^{r} t_i - \lambda (n-r) T + \text{constant} \\
\frac{\partial}{\partial \lambda} \ln \mathcal{L}(\lambda) &= \frac{r}{\lambda} - \sum_{i=1}^{r} t_i - (n-r) T = 0 \\
\hat{\lambda} &= \frac{r}{\sum_{i=1}^{r} t_i + (n-r) T}
\end{aligned}$$

Q4. [10 pts] Decision Trees

Consider the design matrix

$$\begin{bmatrix} 4 & 6 & 9 & 1 & 7 & 5 \\ 1 & 6 & 5 & 2 & 3 & 4 \end{bmatrix}^\top$$

representing 6 sample points, each with two features f_1 and f_2.

The labels for the data are

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}^\top$$

In this question, we build a decision tree of depth 2 by hand to classify the data.

(a) [2 pts] What is the entropy at the root of the tree?

$$-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

(b) [3 pts] What is the rule for the first split? Write your answer in a form like $f_1 \geq 4$ or $f_2 \geq 3$. Hint: you should be able to eyeball the best split without calculating the entropies.

If we sort by f_1, the features and the corresponding labels are

$$\begin{bmatrix} 1 & 4 & 5 & 6 & 7 & 9 \\ 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}.$$

If we sort by f_2, we have

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

The best split is $f_1 \geq 7$.

(c) [3 pts] For each of the two treenodes after the first split, what is the rule for the second split?

For the treenode with labels $(1, 1)$, there's no need to split again.

For the treenode with labels $(0, 1, 0, 0)$, if we sort by f_1, we have

$$\begin{bmatrix} 1 & 4 & 5 & 6 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

If we sort using f_2, we get

$$\begin{bmatrix} 1 & 2 & 4 & 6 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

We easily see we should choose $f_2 \geq 2$.

(d) [2 pts] Let's return to the root of the tree, and suppose we're incompetent tree builders. Is there a (not trivial) split at the root that would have given us an information gain of zero? Explain your answer.

Yes. The rules $f_1 \geq 5$, $f_2 \geq 3$, or $f_2 \geq 5$ would all fail to reduce the weighted average entropy below 1.

Q5. [11 pts] Bagging and Random Forests

We are building a random forest for a 2-class classification problem with t decision trees and bagging. The input is a $n \times d$ design matrix X representing n sample points in $\mathbb{R}^d$ (quantitative real-valued features only). For the ith decision tree we create an n-point training set $X^{(i)}$ through standard bagging. At each node of each tree, we randomly select k of the features (this random subset is selected independently for each treenode) and choose the single-feature split that maximizes the information gain, compared to all possible single-feature splits on those k features. Assume that we can radix sort real numbers in linear time, and we can randomly select an item from a set in constant time.

(a) [3 pts] Remind us how bagging works. How do we generate the data sets $X^{(i)}$? What do we do with duplicate points?

For each training set $X^{(i)}$, we select n sample points from X uniformly at random **with replacement**. Duplicate points have proportionally greater weight in the entropy (or other cost function) calculations. (We will accept an answer that states that duplicate points are treated as if they were separate points infinitesimally close together.)

(b) [3 pts] Fill in the blanks to derive the overall running time to construct a random forest with bagging and random subset selection. Let h be the height/depth (they're the same thing) of the tallest/deepest tree in the forest. You must use the tightest bounds possible with respect to n, d, t, k, h, and n'.

Consider choosing the split at a treenode whose box contains n' sample points. We can choose the best split for these n' sample points in $O($_____$)$ time. Therefore, the running time **per sample point in that node** is $O($_____$)$.

Each sample point in $X^{(i)}$ participates in at most $O($_____$)$ treenodes, so each sample point contributes at most $O($_____$)$ to the time. Therefore, the total running time for one tree is $O($_____$)$.

We have t trees, so the total running time to create the random forest is $O($_____$)$.

The blanks in order: $O(n'k)$, $O(k)$, $O(h)$, $O(kh)$, $O(nkh)$, $O(nkht)$. They are each worth half a point.

(c) [2 pts] If we instead use a support vector machine to choose the split in each treenode, how does that change the asymptotic **query** time to classify a test point?

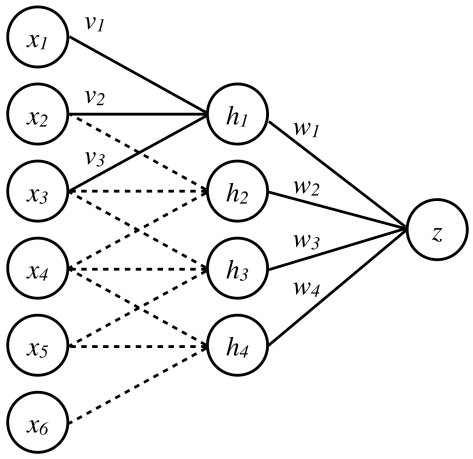
It slows queries down by a factor of $\Theta(d)$ or $\Theta(k)$ (depending whether you run the SVM on k features or all d features—we'll accept either interpretation), because it is necessary to inspect all d (or k) features of the query point at each treenode.

(d) [3 pts] Why does bagging by itself (without random subset selection) tend **not** to improve the performance of decision trees as much as we might expect?

It is common that the same few features tend to dominate in all of the subsets, so almost all the trees will tend to have very similar early splits, and therefore all the trees will produce very similar estimates. The models are not decorrelated enough.

Q6. [11 pts] One-Dimensional ConvNet Backprop

Consider this convolutional neural network architecture.



In the first layer, we have a one-dimensional convolution with a single filter of size 3 such that $h_i = s\left(\sum_{j=1}^{3} v_j x_{i+j-1}\right)$. The second layer is fully connected, such that $z = \sum_{i=1}^{4} w_i h_i$. The hidden units' activation function $s(x)$ is the logistic (sigmoid) function with derivative $s'(x) = s(x)\,(1 - s(x))$. The output unit is linear (no activation function). We perform gradient descent on the loss function $R = (y - z)^2$, where y is the training label for x.

(a) [1 pt] What is the total number of parameters in this neural network? Recall that convolutional layers share weights. There are no bias terms.

The answer is 7. There are 3 parameters in layer 1 and 4 parameters in layer 2.

(b) [4 pts] Compute $\partial R / \partial w_i$.

$$\frac{\partial R}{\partial w_i} = -2(y - z)h_i$$

(c) [1 pt] Vectorize the previous expression—that is, write $\partial R / \partial w$.

$$\frac{\partial R}{\partial w} = -2(y - z)h$$

(d) [5 pts] Compute $\partial R / \partial v_j$.

$$\frac{\partial R}{\partial v_j} = -2(y - z)\frac{\partial z}{\partial v_j} = -2(y - z)\sum_{i=1}^{4}\frac{\partial z}{\partial h_i}\frac{\partial h_i}{\partial v_j} = -2(y - z)\sum_{i=1}^{4} w_i h_i (1 - h_i) x_{i+j-1}$$